# EE5203 L01 Lab Reference

The VS Code + STM32CubeMX workflow and first-day troubleshooting

Basri Erdogan

## Table of contents



This page supports the L01 Lab Guide. It is a reference, not a second required manual.

### The toolchain, by name

You use **two applications** and one invisible toolbox:

| Tool | Job |
|---|---|
| **STM32CubeMX** | select the board, configure pins, generate the C project |
| **VS Code** (STM32 profile) | edit, build, flash, and debug that project |
| STM32CubeCLT (behind the scenes) | the compiler, CMake/Ninja build tools, ST-LINK debug server that VS Code drives |

On laboratory PCs, open VS Code through the **EE5203** desktop shortcut — it selects the course profile with the correct extensions.

### Workflow reference

| Stage | Where / action | Expected evidence |
|---|---|---|
| Create | STM32CubeMX → Board Selector → **NUCLEO-F401RE**; accept default initialization | board name in the project view |
| Configure | Pinout & Configuration tab | PA5 labeled LD2 / GPIO output |
| Generate | Project Manager: name, location (your `C:\Work` folder), **Toolchain/IDE = CMake** → Generate Code | project folder with `Core/Inc`, `Core/Src`, `main.c`, `CMakeLists.txt` |
| Open | VS Code → File → Open Folder → the generated folder | STM32/CMake extension picks up the project (accept the default preset if asked) |

| Stage | Where / action | Expected evidence |
|---|---|---|
| Edit | `Core/Src/main.c`, inside protected `USER CODE` regions | application statements preserved on regeneration |
| Build | **Build** button in the status bar (or Terminal → Run Build Task) | `Build finished` with **0 errors** in the terminal; `.elf` in `build/` |
| Flash + run | press **F5** (Run → Start Debugging) | firmware downloads over ST-LINK; execution stops at `main` |
| Debug | debug toolbar + side panels | execution marker, breakpoints, variables |

**F5 is both the flash and the debug step** — there is no separate "program the board" click in the normal loop. Press F5, then **Continue** to let the program run.

## Debug controls (toolbar, left to right)

| Control | Meaning |
|---|---|
| Continue (F5) | run until the next breakpoint |
| Pause | suspend a running program |
| Step over (F10) | execute the current line without entering called functions |
| Step into (F11) | enter the called function when debug info permits |
| Step out (Shift+F11) | run until the current function returns |
| Restart | reset and run again from the beginning |
| Stop (Shift+F5) | end the debug session |

## Evidence panels (debug side bar)

| Panel | What it shows | Course name |
|---|---|---|
| VARIABLES / WATCH | C variables; add expressions to Watch | variable inspection |
| PERIPHERALS | live peripheral register bits (from the chip's SVD description) | **the SFR view** — register evidence lives here |
| MEMORY | raw memory bytes at an address | Memory view |
| Disassembly View (right-click → Open Disassembly View) | the machine instructions behind your C | Disassembly view |

When a lab guide says *"inspect X in the SFR view"*, it means the **PERIPHERALS** panel.

## Minimal application pattern

```c
/* USER CODE BEGIN 2 */
uint32_t toggle_count = 0;
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    toggle_count++;
    HAL_Delay(500);

    /* USER CODE BEGIN 3 */
```

```
}
/* USER CODE END 3 */
```

Preserve the intended `USER CODE` boundaries — regeneration from the `.ioc` keeps only what is inside them.

## First-day troubleshooting

### Board not detected

- Confirm the board's power LED.
- Use the ST-LINK USB connector and a known **data** cable and port.
- Close other software that may hold ST-LINK (CubeMX programmer window, a second debug session).
- Reconnect the board, then press F5 again.

### Build fails

- Save the edited file (unsaved dot on the tab?).
- Read the **first** relevant error in the terminal and its file/line.
- Check braces, parentheses, semicolons, and names.
- Confirm VS Code has the intended **folder** open (title bar).

### F5 flashes but behavior is unchanged

- Confirm the build actually ran before the flash (watch the terminal).
- Set a breakpoint in the changed code — is it hit?
- Inspect `toggle_count` to determine whether the loop executes.

### Code disappears after regeneration

- Move application changes into protected `USER CODE` regions.
- Treat the `.ioc` as the generated-configuration source of truth: pin changes happen in CubeMX, code changes in VS Code.

### Working on lab PCs

Work in your local `C:\Work\<studentnumber>` folder, never directly on the network drive. At the end of the session run the desktop **Save to H:** shortcut — it copies your project (without `build/`) to your student folder.

## Help request format

Use this template:

```
Expected:
Observed:
Evidence collected:
Smallest next test:
```